

Homework 1 Design Document: In-Memory B+ Tree Index

1. Overview

This program implements a B+ tree index entirely in memory, as specified in the instructor's clarification (no page size, no index file, no disk I/O statistics). Keys and pointers are integers and duplicate keys are rejected. The program is started with "java bplus init <d>" and reads INSERT, DELETE, SEARCH, RANGESEARCH, PRINT and PRINT STATISTICS commands from standard input, printing the exact message formats from the assignment. The algorithms follow Chapter 10 of Ramakrishnan and Gehrke, Database Management Systems, 3rd ed.

2. Data Structures

- Node. A single node class represents both leaves and internal nodes. It stores a boolean leaf flag and an ArrayList of integer keys. Leaves also hold a parallel ArrayList of pointers (ptrs.get(i) belongs to keys.get(i)) and a next reference to the right sibling leaf, forming the linked list used by range queries. Internal nodes hold an ArrayList of child references with $\text{children.size() = keys.size() + 1}$.
- Capacity parameter d. A leaf holds at most $2d$ pairs. An internal node holds at most $2d$ keys and $2d+1$ children. Every non-root node must hold at least d keys (half full). The root is exempt: a leaf root may hold 0 to $2d$ pairs, and an internal root needs at least one key.
- Separator convention. For an internal node, child i contains keys k with $\text{keys}[i-1] \leq k < \text{keys}[i]$. Searches therefore follow child index = number of separator keys $\leq$ the search key.
- BPlusTree. Holds d , the root, and the output writer. An empty tree is a single empty leaf. Output is buffered in a PrintWriter and flushed once at the end.
- Page management. Per the instructor's clarification, nothing is stored on disk, so there are no pages, page size parameter or index file. A node is an ordinary Java object, and the parameter d replaces the page size as the node capacity limit. Because the tree exists only in memory, nothing persists after the program ends.

3. Algorithms

- Search. Descend from the root using the separator rule to a leaf, then binary-search its keys. Prints "<key> found, point is <pointer>" or "<key> not found".
- Insert. The leaf is located first if the key exists, the not-inserted message is printed and nothing changes. Otherwise a recursive insert places the pair in sorted order. When a leaf overflows ($2d+1$ pairs) it is split: the left leaf keeps d pairs, the new right leaf gets $d+1$, the leaf chain is relinked, and the smallest key of the right leaf is copied up. When an internal node overflows ($2d+1$ keys) the middle key is pushed up (moved, not copied), the left node keeps d keys and the right node gets the last d keys with their children. Splits propagate upward through the recursion, if the root splits, a new root with one key is created and the height grows by one.
- Delete. A recursive delete removes the pair from the leaf. Separator keys are not rewritten when a deleted key equals a separator, since the separator remains a valid bound. After returning from

a child, the parent checks whether that child has fewer than d keys. If so it first borrows from the left sibling when that sibling has more than d keys, otherwise from the right sibling under the same condition. For leaves, the borrowed pair moves over and the parent separator is set to the new first key of the right-hand node. For internal nodes, the parent separator rotates down and the sibling's boundary key rotates up. If neither sibling can lend, the node is merged with its left sibling (or with its right sibling if it is the leftmost child), for internal nodes the parent separator is pulled down into the merged node. If the root becomes an internal node with no keys, its only child becomes the new root and the height shrinks.

- Range search. Locate the leaf for k_1 , then follow the next links collecting pairs until a key exceeds k_2 . If nothing is collected, the "no records" message is printed (the method returns null).
- Print and statistics. A breadth-first traversal prints one line per level, each node is shown in brackets, leaves as (key: pointer) and internal nodes as (key). Statistics (height as number of levels, total nodes, total keys) are computed by a level-order traversal.

4. Testing

Test files cover the assignment's sample command sequence and a scenario with a leaf split, a duplicate insert, a redistribution, a merge with root collapse, a failed delete and an empty range search, using $d = 2$. Expected outputs were derived by hand from the textbook algorithms and compared against the program output.

5. Use of Generative AI

I used Generative AI to help break down the assignment and organize my approach. I provided the assignment instructions and asked it to divide the work into smaller sections so I could work through the assignment piece by piece.

For the implementation, I first developed my own pipeline and approach for solving the problem. I provided that approach to the AI and asked it to review it, correct anything that needed to be changed, and suggest improvements where appropriate. I then asked it to provide commented sections for my code so I could clearly understand which parts of the implementation corresponded to each step of my approach and where each piece needed to be implemented so I could implement the code. If what was provided did not make sense or align I either overrode it and reprompted it with the issues I noticed to see if it could rewrite it.

After completing the implementation, I used Generative AI to double-check my code and make sure it aligned with the assignment requirements. I also used it to review my test results and verify my answers based on the tests I ran. Finally, I used Generative AI to proofread my design document and README for spelling, grammar, and clarity.